

Yes. This is the point where we move from learning OCR mechanics to using a modern document-understanding library. For Stage 1.4.2.5.2 — Docling for Scanned PDFs, we'll use your existing scanned PDF directly. We will not manually convert it to scanned\_page.png first. Docling's DocumentConverter can take a PDF directly, and its PDF pipeline supports OCR for scanned/image-based content. (Docling Project)

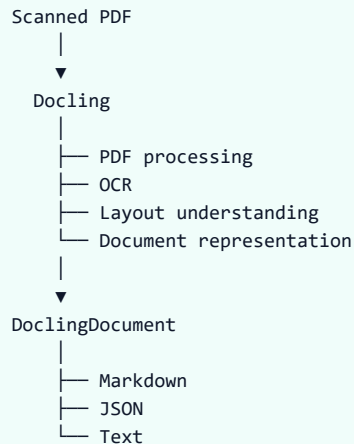
## Stage 1.4.2.5.2 — Docling for Scanned PDFs

### Modern Document-Understanding Pipelines for Scanned Documents in RAG

#### Goal

Our experiment will be:

##### PIPELINE GOAL



This is much closer to how we want to approach document ingestion in a production RAG system.

#### Step 1 — Install Docling

From the root of your existing rag-learning project, run:

##### BASH

```
uv add docling
```

This is the installation command recommended by the current Docling documentation, and Docling supports Windows. (Docling Project)

After installation, verify:

##### BASH

```
uv run python -c "import docling; print('Docling installed successfully')"
```

You should get:

##### OUTPUT

```
Docling installed successfully
```

##### Important

At this stage, don't install pytesseract just for this exercise unless you already installed it from the previous lesson.

Docling itself supports multiple OCR engines, including Tesseract, RapidOCR and EasyOCR. (Docling Project)

For our first Docling experiment, we'll start with Docling's standard PDF pipeline rather than manually wiring an OCR engine.

#### Step 2 — Verify the Notebook Kernel

Open your existing notebook in VS Code.

Make sure the selected kernel is the same Python environment managed by your uv project.

Run:

**PYTHON**

```
import sys

print(sys.executable)
```

This is important because we want:

**ENVIRONMENT RESOLUTION**

```
VS Code Notebook
  ↓
your UV environment
  ↓
Docling
```

and not some unrelated global Python installation.

**Step 3 — Import DocumentConverter**

Now create a new notebook cell:

**PYTHON**

```
from docling.document_converter import DocumentConverter
```

If this imports successfully, our basic Docling environment is ready.

**Step 4 — Define the Scanned PDF**

We are going to use the scanned PDF you already have.

Don't create an image manually.

For example:

**PYTHON**

```
from pathlib import Path

pdf_path = Path("path/to/your/scanned.pdf")

print(pdf_path.exists())
print(pdf_path)
```

We want: True

If you don't remember the exact location, we can identify it from your existing notebook/project files rather than guessing the path.

**Step 5 — Create the Docling Converter**

Now:

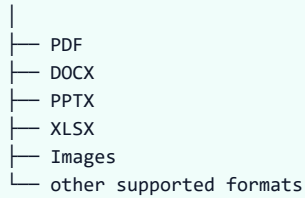
**PYTHON**

```
converter = DocumentConverter()
```

This is the central object we'll use.

Conceptually:

**DOCUMENT CONVERTER SCOPE**

**DocumentConverter**

Docling's DocumentConverter is its primary entry point for converting documents into its unified document representation. (Docling Project)

## Step 6 — Convert the Scanned PDF

Now the important line:

**PYTHON**

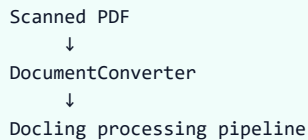
```
result = converter.convert(pdf_path)
```

That's it.

Notice what we didn't write:

- ☐ PDF → PNG manually
- ☐ Pillow
- ☐ pytesseract.image\_to\_string()
- ☐ manually combine pages

Instead:

**STREAMLINED PIPELINE**

## Step 7 — Understand What result Is

Let's inspect:

**PYTHON**

```
print(type(result))
```

You'll get a Docling conversion result object.

Now:

**PYTHON**

```
print(result.status)
```

And:

**PYTHON**

```
print(result.document)
```

The important object is: `result.document`

This is a DoclingDocument.

Conceptually:

**RESULT COMPOSITION**

```

result
├── status
├── input information
└── document
    └── DoclingDocument

```

## Step 8 — Export the Result as Markdown

Now let's see what Docling understood.

### PYTHON

```
markdown_text = result.document.export_to_markdown()
```

Display it:

### PYTHON

```
print(markdown_text)
```

This is where things become interesting.

Instead of getting only raw OCR text, we're asking Docling to represent the processed document in Markdown.

Docling's standard usage explicitly supports converting a document and exporting the resulting DoclingDocument to Markdown. (Docling Project)

## Step 9 — Save the Markdown

Let's preserve the result:

### PYTHON

```

markdown_path = Path("scanned_document_docling.md")

markdown_path.write_text(
    markdown_text,
    encoding="utf-8"
)

print(f"Saved: {markdown_path}")

```

Now we have:

### OUTPUT ARTIFACT

```

scanned.pdf
├──
└──
    └── Docling
        └──
            scanned_document_docling.md

```

## Step 10 — Export as JSON

Now let's look at another important representation.

### PYTHON

```
json_text = result.document.export_to_json()
```

Save it:

### PYTHON

```

json_path = Path("scanned_document_docling.json")

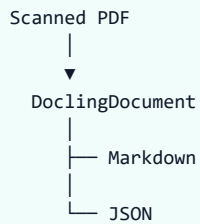
json_path.write_text(
    json_text,
    encoding="utf-8"
)

print(f"Saved: {json_path}")

```

Now:

#### UNIVERSAL EXPORT ARTIFACTS



Docling's newer document model is specifically designed as a universal representation that can preserve document hierarchy and can be exported to formats including JSON and Markdown. (Docling Project)

## Step 11 — Inspect the JSON

Let's see what we have.

#### PYTHON

```

import json

doc_json = json.loads(json_text)

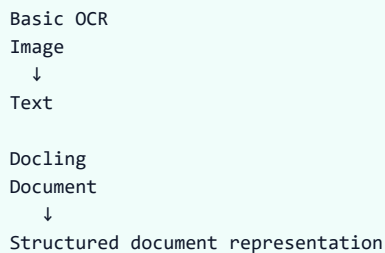
print(type(doc_json))
print(doc_json.keys())

```

Don't worry if the exact keys differ from what you expect.

The important conceptual difference is:

#### CONCEPTUAL DIFFERENCE



That distinction is extremely important for your RAG journey.

## Step 12 — Compare Basic OCR vs Docling

Now let's explicitly compare the two approaches.

#### APPROACH A — TESSERACT

Approach A – Basic Tesseract

Scanned PDF

↓

Page Image

↓

Tesseract

↓

Plain Text

Output:

"Employee Details Name Department..."

APPROACH B — DOCLING

Approach B – Docling

Scanned PDF

↓

Docling

↓

OCR + document processing

↓

DoclingDocument

↓

Markdown / JSON / other representations

Potentially:

| Name   | Department | Experience |
|--------|------------|------------|
| Ravi   | Data       | 8          |
| Suresh | IT         | 10         |

The second representation is much more useful for downstream RAG processing. Docling's PDF pipeline also supports table-structure processing, which is one reason it is more appropriate for complex documents than treating everything as plain OCR text. (Docling Project)

Step 13 — Check Whether OCR Actually Happened

This is an important experiment. Our input is a scanned PDF. Therefore, we want to see whether Docling was able to recover text from the page. Start with:

PYTHON

print(markdown\_text[:3000])

Look at the output. If you see meaningful text from the scanned document, Docling successfully processed the scanned content.

Step 14 — Understand What Docling Is Doing for Us

This is the key learning point. Previously:

PREVIOUS WORKFLOW

```

PDF
↓
render page
↓
PNG
↓
Tesseract
↓
text
    
```

Now:

#### DOCLING UNIFIED WORKFLOW

```

PDF
↓
Docling PDF pipeline
↓
document processing
├─ OCR
├─ layout
├─ reading order
├─ table structure
└─ document representation
↓
DoclingDocument
    
```

That's why I recommended moving toward Docling.

### Step 15 — But Don't Assume Docling Solves Everything

This is important for your production-grade RAG learning.

Docling is not magic.

For example:

#### EXAMPLE A

Scanned architecture diagram  
is different from:  
Scanned paragraph

And:

#### EXAMPLE B

Mathematical formula  
is different from:  
Table

Docling has additional enrichment capabilities and different processing pipelines for more advanced scenarios. Its current CLI, for example, exposes options for formula enrichment, picture description, chart extraction and other processing capabilities. (Docling Project)

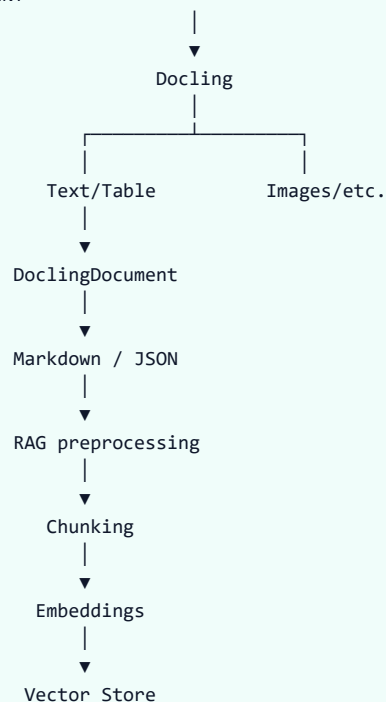
We'll learn those after we understand the basic Docling pipeline.

### Step 16 — Our First RAG-Oriented Architecture

At this point, our architecture becomes:

#### RAG-ORIENTED INGESTION ARCHITECTURE

## DOCUMENT



This is much closer to the architecture we'll eventually use in your production-grade RAG.

## Step 17 — What We Are NOT Doing Yet

For this stage, don't add:

- custom OCR preprocessing
- OpenCV
- image thresholding
- manual Tesseract configuration
- table extraction tuning
- formula extraction
- chart extraction
- VLM pipelines
- LangChain integration
- chunking

Those belong to later stages.

We're learning one thing now: How can Docling ingest a scanned PDF and turn it into a structured document representation?

## Step 18 — Your Notebook Cells for This Stage

You can keep the implementation very small.

### CELL 1 — IMPORT

```
from pathlib import Path
from docling.document_converter import DocumentConverter
```

### CELL 2 — PDF

```
pdf_path = Path("path/to/your/scanned.pdf")

print("Exists:", pdf_path.exists())
print("Path:", pdf_path)
```

### CELL 3 — CONVERTER



```
converter = DocumentConverter()
```

#### CELL 4 — CONVERT

```
result = converter.convert(pdf_path)

print("Status:", result.status)
```

#### CELL 5 — GET DOCLING DOCUMENT

```
doc = result.document

print(type(doc))
```

#### CELL 6 — MARKDOWN

```
markdown_text = doc.export_to_markdown()

print(markdown_text)
```

#### CELL 7 — SAVE MARKDOWN

```
Path("scanned_document_docling.md").write_text(
    markdown_text,
    encoding="utf-8"
)

print("Markdown saved.")
```

#### CELL 8 — JSON

```
json_text = doc.export_to_json()

Path("scanned_document_docling.json").write_text(
    json_text,
    encoding="utf-8"
)

print("JSON saved.")
```

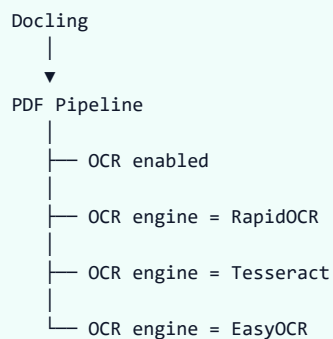
## Step 19 — One Important Thing About OCR Configuration

There's a subtle point here.

Docling's current PDF pipeline supports OCR, but its OCR configuration is customizable. The documentation exposes PdfPipelineOptions, including do\_ocr and OCR engine options. (Docling Project)

So eventually we can explicitly configure:

#### OCR ENGINE CONFIGURATION OPTIONS



For this first experiment, however, I want you to see what the standard Docling pipeline gives us before we start changing its internals.

## Step 20 — What You've Achieved

This is the important transition in your RAG journey:

### BEFORE

You were learning:  
How does OCR work?

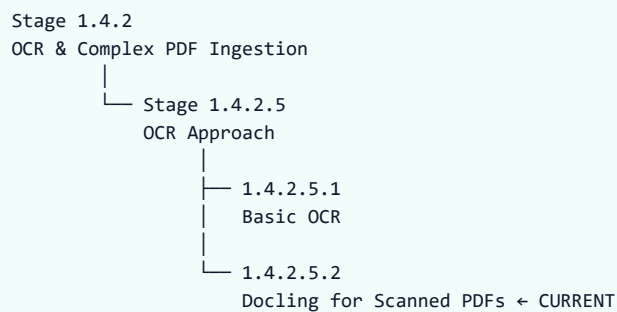
### NOW

You're learning:  
How do modern document-processing frameworks  
handle OCR and complex documents for RAG?

And that's much closer to your actual goal.

Your current journey is now:

### LEARNING ROADMAP



## Next Actions

Run Steps 1–8 first. In particular, after `result = converter.convert(pdf_path)`, inspect the Markdown output from your actual scanned PDF. That output will determine what we investigate next rather than blindly moving ahead.